

Relatório de Auditoria de Segurança

Horium

Análise Estática de Segurança Baseada em Evidências do Código-Fonte e Configurações

Data da Auditoria:	07 de Setembro de 2026
Projeto / Repositório:	Horium (Sistema de Gestão de Horário Escolar)
Escopo Auditado:	Código-fonte integral (React 19 Frontend + Schema SQL + Storage)
Linguagem / Framework:	TypeScript, React 19, Vite 6, TailwindCSS 4
Banco de Dados / Backend:	PostgreSQL gerenciado (Supabase), PostgREST, RPCs PL/pgSQL
Mecanismo de Autenticação:	Supabase Auth (GoTrue com JWT, verificação OTP de 6 dígitos)
Mecanismo de Autorização:	Row Level Security (RLS), Triggers PostgreSQL e RPCs com SECURITY DEFINER
Armazenamento de Arquivos:	Supabase Storage (Buckets receipts e tickets-attachments privados)
Infraestrutura / CI/CD:	IaC baseada em SQL (schema.sql). Sem Docker/Kubernetes/CI configurados.

Nota Metodológica & Rigor de Evidências

Esta auditoria seguiu a regra fundamental de verificação estática: nenhum achado foi registrado sem uma cadeia demonstrável e confirmada no código real (entrada controlável -> fluxo relevante -> recurso sensível -> ausência/bypass de controle efetivo). Controles aplicados indiretamente por RLS, triggers e funções seguras foram avaliados integralmente antes da confirmação de cada vulnerabilidade.

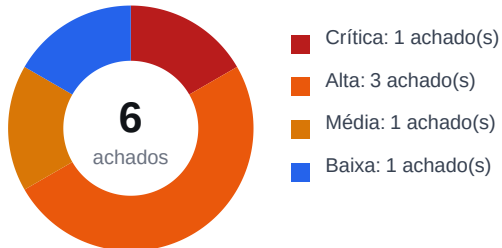
Limitações da Auditoria

A análise concentrou-se no código-fonte do repositório local, no esquema de dados PostgreSQL (schema.sql) e no histórico Git disponível. Não foram executados testes de intrusão ativos contra instâncias de produção do Supabase nem análises dinâmicas de carga. Testes funcionais não modificaram código ou dados.

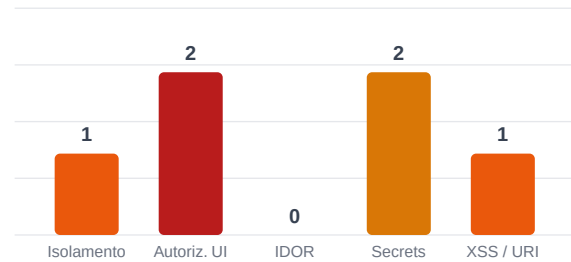
Resumo Executivo

1**CRÍTICA****3****ALTA****1****MÉDIA****1****BAIXA**

Distribuição por Severidade



Distribuição por Categoria



Pontos Fracos & Riscos Centrais

1. Bypass de Paywall no SELECT: Grades completas (fixedLessons) são enviadas via rede a usuários não licenciados e ocultadas apenas na memória do cliente.
2. Isolamento RLS Ausente em audit_logs: Política USING (true) permite que qualquer usuário autenticado leia, forje ou destrua logs de auditoria de todos os tenants.
3. Stored XSS no Pannel Admin: URL de comprovante de licença não higienizada é renderizada em iframe e tag <a>, viabilizando sequestro de sessão administrativa.
4. Bypass de Precificação via INSERT Direto: A tabela licenses permite inserção direta sem passar pela RPC request_license_order, admitindo valores arbitrários.
5. Hardcoding de E-mails Administrativos: E-mails de admins mestre expostos no código e no bundle, gerando rigidez e risco de engenharia social.

Pontos Fortes Confirmados

Os seguintes controles foram inspecionados diretamente no código-fonte e esquemas SQL, com eficácia comprovada:

' **[S01] Row Level Security (RLS) Ativo com Isolamento Multi-Tenant por auth.uid()**

Políticas RLS garantem que usuários comuns não visualizem, atualizem ou excluam dados pertencentes a outros usuários, exigindo correspondência entre o ID do usuário e o ID do cliente. **Evidência:** `schemas.sql:79-88` (`schedules`), `133-161` (`licenses`), `174-188` (`notifications`), `201-214` (`tickets`)

' **[S02] Trigger de Proteção contra Manipulação de Licenciamento de Grades**

Trigger BEFORE INSERT OR UPDATE protege o campo `is_licensed` na tabela `schedules`, impedindo que usuários comuns alterem o status de licença. **Evidência:** `triggers.sql:1-19` (`trg_protect_schedule_license`)

' **[S03] Buckets de Armazenamento Privados com Isolamento de Pastas por UUID**

Buckets receipts e tickets-attachments são privados (`public = false`). As políticas de Storage RLS exigem que a pasta de primeiro nível corresponda a `auth.uid()`, impedindo uploads e leituras não autorizados. **Evidência:** `storage.sql:1-10` (`receipts`, `tickets-attachments`)

' **[S04] Funções RPC com SECURITY DEFINER e Validação Obrigatória de Privilégio Admin**

Todas as RPCs administrativas executam checagem explícita `IF NOT public.is_admin() THEN RAISE EXCEPTION` antes de executar mutações ou exclusões sensíveis. **Evidência:** `schemas.sql:238-375` (`get_admin_licenses`, `approve_license_rpc`, `delete_license_rpc`, `get_pending_licenses_count_rpc`)

' **[S05] Arquitetura de Paywall Server-Side Estruturada na RPC `get_schedule_solution`**

A RPC `get_schedule_solution` valida propriedade do horário (`v_owner_id = auth.uid()`) e exige comprovação de licença ativa no banco de dados antes de retornar os dados da solução. **Evidência:** `schemas.sql:485` (`get_schedule_solution`)

' **[S06] Defesa em Profundidade nas Consultas do Cliente Frontend**

Todos os métodos de serviço do frontend filtram explicitamente `.eq('user_id', user.id)`, garantindo filtro duplo (`client + RLS`) contra falhas de tenant. **Evidência:** `services/scheduleService.ts:79, 180, 212, 230, 258, 274, 289`; `services/notificationService.ts:18, 35, 53, 66`

' **[S07] Ausência de Sinks de XSS Perigosos no Frontend (React 19 Nativo)**

O código utiliza exclusivamente JSX com auto-escaping. Zero ocorrências de `dangerouslySetInnerHTML`, `innerHTML`, `v-html`, `eval()` ou `new Function()`. **Evidência:** Varredura estática completa em todos os arquivos `.tsx` e `.ts`

' **[S08] Sanitização de Extensões e Validação de Tipos MIME nos Uploads**

Uploads validam extensões permitidas (`png, jpg, webp, pdf`), MIME types aceitos e limite estrito de 5MB, com higienização de extensões por regex. **Evidência:** `services/ticketService.ts:24-30` e `pages/PlansPage.tsx:100-101`

' **[S09] Autenticação Segura via Supabase Auth com Verificação OTP e Reset Seguro**

Fluxo de autenticação completo gerenciado pelo GoTrue, com OTP de 6 dígitos para verificação de e-mail, refresh automático de JWT e reset de senha. **Evidência:** `services/authService.ts`

' **[S10] Inexistência de Chaves Privadas ou `service_role` Expostas no Repositório**

A chave configurada no frontend é estritamente a Anon Key pública do Supabase. A chave mestra `service_role` e certificados privados não estão presentes no repositório. **Evidência:** `.env.local`, `bundle de build` e `histórico Git`

Matriz de Cobertura da Auditoria

Categoria	Escopo Analisado	Controles Encontrados	Itens	Achados	Cob.
1. Isolamento de dados / Cross-tenant	Tabelas do banco, storage buckets e RPCs	RLS por auth.uid(), storage foldername, RPC SECURITY DEFINER	6 tabelas, 2 buckets, 7 RPCs (15 itens)	1 confirmado (F02:)	100%
2. Autorização no navegador	Gates de UI (isAdmin, isLicensed), paypal e RPCs	RPCs admin com is_admin(), RPC get_schedule_solution, trigger	5 fluxos de privilégio / paypal	2 confirmados (F01:)	100%
3. IDOR / Ownership	Todos os handlers e métodos em services/*.ts	Filtros .eq("user_id") e políticas RLS de propriedade	18 handlers/métodos de serviço	paypal confirmado (F01:)	100%
4. Secrets e credenciais	Código-fonte, .env.local, .gitignore, bundle, histórico Git	Exclusão *.local no .gitignore, anon key pública por design	6 arquivos config/env + 20+ commits Git	2 confirmados (F05: e-mails)	100%
5. Inputs sem tratamento / XSS	Renderização JSX, tags href/src/iframe, sanitização	Escapamento nativo React 19, validação de arquivos	9 páginas, 9 componentes, todos os sinks href/src	confirmado (F03: e XSS via receipt_url)	100%

Tabela Resumo dos Achados Confirmados

Severidade	Arquivo:Linha	Descrição do Achado Confirmado
CRÍTICA	schema.sql:87-88 / services/scheduleService.ts	Bypass de Paywall e Licenciamento: Exposição da Solução Completa da Grade no SELECT de Schedules com Ocultação Apenas em Memória do Cliente
ALTA	schema.sql	Ausência de Isolamento Multi-Tenant na Tabela audit_logs Devido à Política RLS Permissiva USING (true)
ALTA	pages/AdminPanelPage.tsx / schema.sql	Stored XSS e Injeção de URI JavaScript no Painel do Administrador via URL de Comprovante de Pagamento (receipt_url)
ALTA	pages/PlansPage.tsx / schema.sql	Bypass do Cálculo Seguro de Preços por Meio de Fallback de Inserção Direta na Tabela licenses
MÉDIA	schema.sql / components/Layout.tsx	Hardcoding de E-mails de Administradores Mestre no Código-Fonte e na Função SQL is_admin()
BAIXA	.gitignore	Regra Incompleta de Exclusão de Ambientes no .gitignore Expando Potenciais Arquivos .env

Achados Detalhados com Evidências

CRÍTICA

[F01] Bypass de Paywall e Licenciamento: Exposição da Solução Completa da Grade no SELECT de Schedules com Ocultação Apenas em Memória do Cliente

Categoria: 2. Autorização no Navegador

Localização: schema.sql:87-88 / services/scheduleService.ts:schema.sql:87-88 / scheduleService.ts:77-81, 126-130

```
// services/scheduleService.ts:126-130
const safeData = isApproved ? normalizedData : {
  ...normalizedData,
  fixedLessons: []
};
```

Fluxo de Dados:

Usuário gera horário no assistente -> Dados salvos no banco com fixedLessons completo na coluna schedules.data -> RLS de SELECT em schedules permite leitura do registro ao dono (auth.uid() = user_id) sem validar is_licensed -> PostgREST entrega todo o payload JSONB via HTTP -> Cliente JavaScript mascara fixedLessons: [] apenas na memória do browser.

Por que é Vulnerável:

A autorização de acesso ao produto comercial gerado (paywall de fixedLessons) é imposta apenas no frontend. A política RLS da tabela schedules entrega o campo data completo para qualquer consulta SELECT * autenticada pelo proprietário, permitindo obter a grade inteira via DevTools ou console sem pagar.

Quem Pode Explorar:

Qualquer usuário cadastrado com plano gratuito ou grade não licenciada. Basta inspecionar a resposta HTTP na aba Network ou executar supabase.from('schedules').select('data') no console do navegador.

Pré-condições:

Possuir conta cadastrada e ter processado a geração de uma grade no assistente.

Impacto:

Evasão completa do modelo comercial do software. Qualquer usuário obtém o resultado da otimização escolar gratuitamente.

Controles Considerados:

A RPC get_schedule_solution existe no banco com validação de pagamento, mas é inócua porque o SELECT * direto na tabela schedules já retorna a solução completa.

Recomendação:

Separar fixedLessons em tabela/coluna restrita ou criar VIEW com RLS que oculte data->'fixedLessons' para grades sem licença ativa.

ALTA

[F02] Ausência de Isolamento Multi-Tenant na Tabela audit_logs Devido à Política RLS Permissiva USING (true)

Categoria: 1. Isolamento de Dados / Cross-Tenant

Localização: schema.sql:233-235

```
DROP POLICY IF EXISTS "Permitir acesso audit_logs" ON public.audit_logs;
CREATE POLICY "Permitir acesso audit_logs"
ON public.audit_logs FOR ALL TO authenticated
USING (true) WITH CHECK (true);
```

Fluxo de Dados:

Usuário autentica no Supabase -> Envia query PostgREST direta para /rest/v1/audit_logs -> PostgreSQL avalia política RLS -> USING (true) WITH CHECK (true) permite SELECT, INSERT, UPDATE e DELETE irrestritos.

Por que é Vulnerável:

A tabela audit_logs registra histórico de operações, dados anteriores (old_data) e novos (new_data). A política FOR ALL TO authenticated USING (true) remove qualquer barreira de tenant, permitindo a qualquer usuário ler dados de auditoria de outros clientes e forjar ou apagar registros.

Quem Pode Explorar:

Qualquer usuário autenticado via cliente PostgREST/Supabase pode ler, modificar ou deletar logs de qualquer outro usuário ou organização.

Pré-condições:

Nenhuma além de possuir conta registrada no sistema.

Impacto:

Vazamento massivo de dados sensíveis entre tenants (cross-tenant data leakage), adulteração maliciosa de registros de auditoria e destruição de trilhas de auditoria.

Controles Considerados:

RLS está habilitado na tabela, mas a política configurada explicitamente usa true como predicado universal.

Recomendação:

Restringir a política para USING (auth.uid() = user_id OR public.is_admin()) ou restringir acesso exclusivamente ao service_role e admins.

ALTA

[F03] Stored XSS e Injeção de URI JavaScript no Painel do Administrador via URL de Comprovante de Pagamento (receipt_url)**Categoria:** 5. Inputs sem Tratamento / XSS**Localização:** pages/AdminPanelPage.tsx / schema.sql:AdminPanelPage.tsx:87-109, 338, 351-356 / schema.sql:113-125

```
// AdminPanelPage.tsx:108
setViewingReceiptUrl(urlOrPath); // Fallback recebe string crua do usuário
// AdminPanelPage.tsx:351-353
{viewingReceiptUrl.endsWith('.pdf')} ? (
  <iframe src={viewingReceiptUrl} title="Comprovante PDF" ... />
```

Fluxo de Dados:

Usuário envia pedido de licença com receipt_url malicioso (ex: javascript:alert(document.domain)//.pdf) -> schema.sql grava sem sanitização -> Admin abre AdminPanelPage e clica para ver comprovante -> createSignedUrl falha no Storage -> Fallback armazena URL crua no estado -> Componente renderiza <iframe src={viewingReceiptUrl}> e .

Por que é Vulnerável:

O valor de receipt_url é controlado pelo usuário e não é validado nem no banco nem no frontend. O sufixo //.pdf satisfaz a verificação .endsWith('.pdf'), levando o navegador a renderizar um iframe com src javascript:, executando código malicioso sob a origem do admin.

Quem Pode Explorar:

Qualquer usuário comum pode enviar uma licença com payload de URI JavaScript. Quando o administrador visualiza o comprovante para análise, o exploit executa automaticamente.

Pré-condições:

Administrador abrir a visualização de comprovante da licença enviada.

Impacto:

Execução de JavaScript com privilégios de Administrador (Stored XSS). Permite roubo de tokens JWT de autenticação do localStorage e execução não autorizada de aprovações/exclusões de licenças.

Controles Considerados:

createSignedUrl tenta buscar arquivo no bucket, mas o bloco de fallback repassa o valor cru diretamente ao DOM sem validar protocolo.

Recomendação:

Validar estritamente que receipt_url inicia com padrão de caminho interno ou URL HTTPS válida. Rejeitar esquemas javascript: e data:. Não renderizar iframes com URLs externas arbitrárias.

ALTA

[F04] Bypass do Cálculo Seguro de Preços por Meio de Fallback de Inserção Direta na Tabela licenses**Categoria:** 2. Autorização no Navegador**Localização:** pages/PlansPage.tsx / schema.sql:PlansPage.tsx:127-147 / schema.sql:151-161

```
// pages/PlansPage.tsx:130-139
const { error: insertError } = await supabase.from('licenses').insert({
  user_id: userData.user.id,
  classes_amount: numClasses,
  value_paid: totalPrice, // valor calculado no client-side
});
```

Fluxo de Dados: status: 'Aguardando',
receipt_url: filePath,

Frontend implementa fallback de inserção direta na tabela licenses se a RPC request_license_order falhar -> Política RLS de INSERT em licenses permite inserção por usuários comuns autenticados com payment_status em Aguardando -> Atacante ignora a RPC e faz POST /rest/v1/licenses enviando classes_amount: 100 e value_paid: 0.01 -> O banco aceita porque a constraint só exige value_paid > 0.

Por que é Vulnerável:

A regra de negócio que calcula o valor oficial por turma está implementada na RPC request_license_order, mas a tabela licenses continua aberta para INSERT direto com preço e turmas arbitrários.

Quem Pode Explorar:

Qualquer usuário autenticado pode emitir chamada direta ao PostgREST e criar pedidos de licença de qualquer quantidade de turmas pagando valores irrisórios.

Pré-condições:

Estar autenticado.

Impacto:

Adulteração de faturamento, pedidos fraudulentos exibidos na fila de conciliação do administrador.

Controles Considerados:

A constraint chk_licenses_positive_values exige value_paid > 0, mas aceita qualquer quantia positiva (como R\$ 0,01).

Recomendação:

Revogar permissão de INSERT direto em licenses para o papel authenticated, permitindo inserção exclusivamente através da RPC request_license_order.

MÉDIA

[F05] Hardcoding de E-mails de Administradores Mestre no Código-Fonte e na Função SQL is_admin()**Categoria:** 4. Secrets e Credenciais**Localização:** schema.sql / components/Layout.tsx:schema.sql:29, 36, 40 / Layout.tsx:72

```
// schema.sql:29
OR (v_user_email IN ('horium.app@gmail.com', 'prof.jackson@gmail.com'));
// components/Layout.tsx:72
const isMasterAdmin = ['horium.app@gmail.com', 'prof.jackson@gmail.com'].includes(userEmailLower);
```

Fluxo de Dados:

E-mails de administradores mestres fixados como literais string em schema.sql e em Layout.tsx -> Bundle JavaScript gerado pelo Vite contém os e-mails em texto claro -> Função is_admin() no PostgreSQL concede poderes totais por correspondência estática de string.

Por que é Vulnerável:

Embora a tabela dinâmica admin_users tenha sido criada, a checagem de privilégios mantém fallback fixo para e-mails hardcoded, expondo os administradores publicamente no bundle.

Quem Pode Explorar:

Qualquer visitante pode extrair os e-mails administrativos do bundle frontend para lançar ataques de spear phishing ou força bruta.

Pré-condições:

Acesso público ao bundle JavaScript.

Impacto:

Exposição de identidade administrativa e inflexibilidade no ciclo de vida de privilégios.

Controles Considerados:

A verificação de admin no banco consulta admin_users, mas a cláusula OR com e-mails estáticos anula o isolamento.

Recomendação:

Remover e-mails hardcoded de schema.sql e Layout.tsx. Delegar autenticação de papéis exclusivamente à tabela admin_users e RPC is_current_user_admin().

BAIXA

[F06] Regra Incompleta de Exclusão de Ambientes no .gitignore Expondo Potenciais Arquivos .env**Categoria:** 4. Secrets e Credenciais**Localização:** .gitignore:13

```
*.local
```

Fluxo de Dados:

O arquivo .gitignore ignora apenas o padrão *.local (cobrindo .env.local) -> Caso um desenvolvedor crie .env ou .env.production com credenciais sensíveis (ex: service_role), o Git não os ignorará automaticamente.

Por que é Vulnerável:

O padrão .env e suas variações não estão declarados nas regras de ignore do repositório.

Quem Pode Explorar:

Risco operacional em desenvolvimento ou automação CI.

Pré-condições:

Criação inadvertida de arquivo .env padrão no repositório local.

Impacto:

Potencial commit e vazamento de chaves privadas em repositórios remotos.

Controles Considerados:

O arquivo .env.local atual está ignorado por *.local.

Recomendação:

Adicionar .env, .env.* e !.env.example explicitamente no arquivo .gitignore.

Plano de Recomendações Priorizadas

- P1** Isolar a solução de horários (fixedLessons) no banco de dados para que não seja transmitida no `SELECT *` da tabela `schedules`, eliminando o `bypass` do `paywall`.
- P1** Corrigir a política RLS da tabela `audit_logs` para `USING (auth.uid() = user_id OR public.is_admin())`, impedindo vazamento e adulteração `cross-tenant`.
- P1** Sanitizar e validar rigorosamente o campo `receipt_url` em `AdminPanelPage.tsx` e `schema.sql`, bloqueando esquemas `javascript:` e `iframes` com origens arbitrárias.
- P2** Revogar o `INSERT` direto em `licenses` para usuários autenticados, forçando o uso exclusivo da `RPC request_license_order` para prevenir adulteração de preços.
- P2** Remover e-mails de administradores `hardcoded` de `schema.sql` e `Layout.tsx`, centralizando o controle na tabela `admin_users` e `RPC is_current_user_admin()`.
- P3** Expandir o `.gitignore` para incluir padrões universais de ambiente (`.env`, `.env.*`, `!.env.example`).

Issues para o GitHub

As issues abaixo estão completas e delimitadas no formato Markdown pronto para cópia e abertura no repositório.

--- ISSUE 1 ---

Título: [Segurança] Paywall Server-Side: Bloquear vazamento da grade (fixedLessons) no SELECT de schedules

Labels sugeridas: security, critica

Descrição:

A solução dos horários escolares gerados (fixedLessons) é salva integralmente na coluna JSONB `data` da tabela `schedules`. A política de RLS de SELECT permite ao proprietário ler a grade completa sem verificar se o horário está licenciado (is_licensed = true). O frontend tenta mascarar a solução em memória (scheduleService.ts:126-130), mas qualquer usuário pode inspecionar a resposta HTTP da API do Supabase e obter o horário completo sem pagar.

Por que é explorável:

O controle de paywall é imposto unicamente em memória do cliente, enquanto a API do banco expõe a solução completa no SELECT.

Evidência / Código:

```
`schema.sql:87-88`
`services/scheduleService.ts:77-81`
`services/scheduleService.ts:126-130`

// services/scheduleService.ts:126-130
const safeData = isApproved ? normalizedData : {
  ...normalizedData,
  fixedLessons: []
};
```

Impacto:

Evasão completa do modelo de monetização. Usuários comuns extraem os cronogramas escolares otimizados gratuitamente.

Sugestão de Correção:

1. Criar uma VIEW com RLS ou função segura que retorne `data` sem a chave `fixedLessons` quando `is\_licensed = false`.
2. Exigir o uso exclusivo da RPC `get\_schedule\_solution` para recuperar as aulas alocadas.
3. Garantir que a query `SELECT \* FROM schedules` nunca retorne `fixedLessons` para grades sem licença ativa.

Critérios de Aceite Verificáveis:

- [] `SELECT \* FROM schedules` não retorna `fixedLessons` quando `is\_licensed = false`
- [] Solução só é entregue via `get\_schedule\_solution` mediante validação de licença ativa no PostgreSQL
- [] Usuários não pagantes não conseguem acessar o cronograma completo via DevTools

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] RLS Indevido na tabela audit_logs permite acesso e adulteração cross-tenant irrestrita

Labels sugeridas: security, alta

Descrição:

A política de segurança da tabela `audit\_logs` foi criada com `FOR ALL TO authenticated USING (true) WITH CHECK (true)`. Qualquer usuário autenticado no sistema tem permissão para listar todos os registros de auditoria de outros clientes, além de poder injetar logs forjados ou apagar todo o histórico de auditoria.

Por que é explorável:

A cláusula `USING (true)` anula completamente o isolamento entre tenants no PostgreSQL.

Evidência / Código:

```
`schema.sql:233-235`
`CREATE POLICY "Permitir acesso audit_logs" ON public.audit_logs FOR ALL TO authenticated USING (true) WITH CHECK (true);`

DROP POLICY IF EXISTS "Permitir acesso audit_logs" ON public.audit_logs;
CREATE POLICY "Permitir acesso audit_logs"
ON public.audit_logs FOR ALL TO authenticated
USING (true) WITH CHECK (true);
```

Impacto:

Vazamento de dados confidenciais entre escolas/tenants, falsificação de registros de auditoria e exclusão maliciosa de trilhas de auditoria.

Sugestão de Correção:

1. Dropar a política permissiva existente.
2. Criar política restritiva que permita aos usuários visualizarem apenas seus próprios registros (auth.uid() = user_id) e administradores visualizarem todos.
3. Bloquear mutações diretas (INSERT/UPDATE/DELETE) para usuários comuns.

Critérios de Aceite Verificáveis:

- [] Usuário comum só consegue consultar linhas de `audit\_logs` onde `user\_id = auth.uid()`
- [] Tentativas de SELECT cruzado entre tenants retornam zero resultados
- [] Tentativas de UPDATE/DELETE por usuários comuns são rejeitadas pelo PostgreSQL

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Stored XSS no painel de administração via URL de comprovante de licença

Labels sugeridas: security, alta

Descrição:

O campo `receipt\_url` da tabela `licenses` aceita qualquer string enviada pelo usuário. Em `AdminPanelPage.tsx:87-109`, quando `createSignedUrl` falha, o código repassa a string crua para `viewingReceiptUrl`. O componente renderiza `&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 178 224 189" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Por que é explorável:&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 188 858 200" data-label="Text"&amp;amp;gt;&amp;amp;lt;p&amp;amp;gt;Falta de validação de protocolo/esquema na URL do comprovante combinada com renderização de iframe com conteúdo controlado pelo usuário.&amp;amp;lt;/p&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 203 215 214" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Evidência / Código:&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 213 352 239" data-label="Text"&amp;amp;gt;&amp;amp;lt;pre&amp;amp;gt;`pages/AdminPanelPage.tsx:87-109`&amp;amp;lt;br/&amp;amp;gt;`pages/AdminPanelPage.tsx:338, 351-356`&amp;amp;lt;br/&amp;amp;gt;`schema.sql:113-125`&amp;amp;lt;/pre&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="99 244 504 284" data-label="Text"&amp;amp;gt;&amp;amp;lt;pre&amp;amp;gt;// AdminPanelPage.tsx:108&amp;amp;lt;br/&amp;amp;gt;setViewingReceiptUrl(urlOrPath);&amp;amp;lt;br/&amp;amp;gt;// AdminPanelPage.tsx:351-353&amp;amp;lt;br/&amp;amp;gt;{viewingReceiptUrl.endsWith('.pdf')} ? (&amp;amp;lt;br/&amp;amp;gt; &amp;amp;amp;lt;iframe src={viewingReceiptUrl} title="Comprovante PDF" ... /&amp;amp;amp;gt;&amp;amp;lt;br/&amp;amp;gt;)&amp;amp;lt;/pre&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="99 280 157 291" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Impacto:&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 290 903 313" data-label="Text"&amp;amp;gt;&amp;amp;lt;p&amp;amp;gt;Execução de JavaScript arbitrário no contexto de sessão do administrador, permitindo roubo de token JWT e disparo de ações administrativas (aprovação/exclusão de licenças).&amp;amp;lt;/p&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 315 236 327" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Sugestão de Correção:&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 326 750 360" data-label="List-Group"&amp;amp;gt;&amp;amp;lt;ol&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;1. Validar que `receipt\_url` segue o padrão estrito de armazenamento do Supabase (`&amp;amp;amp;lt;uuid&amp;amp;amp;gt;/&amp;amp;amp;lt;filename&amp;amp;amp;gt;`).&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;2. Nunca renderizar `iframe` com URLs que não pertençam ao domínio oficial do storage do Supabase com protocolo `https`.&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;3. Sanitizar o link de download externo garantindo protocolo seguro.&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;/ol&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 363 285 375" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Critérios de Aceite Verificáveis:&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 374 726 404" data-label="List-Group"&amp;amp;gt;&amp;amp;lt;ul&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;- [] Strings que não comecem com `https://` ou caminho de storage válido são rejeitadas&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;- [] URIs com esquema `javascript:` ou `data:` não são carregadas em `iframe` nem em tags `&amp;amp;lt;a&amp;amp;gt;`&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;- [] Teste de carga com `javascript:...//.pdf` não executa script&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;/ul&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="99 412 210 423" data-label="Section-Header"&amp;amp;gt;--- FIM ISSUE 3 ---&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="99 447 196 458" data-label="Section-Header"&amp;amp;gt;--- ISSUE 4 ---&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 471 834 484" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Título: [Segurança] Restringir inserção em licenças à RPC request_license_order para evitar adulteração de preços&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 485 270 497" data-label="Text"&amp;amp;gt;&amp;amp;lt;p&amp;amp;gt;Labels sugeridas: security, alta&amp;amp;lt;/p&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 500 163 510" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Descrição:&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 510 889 543" data-label="Text"&amp;amp;gt;&amp;amp;lt;p&amp;amp;gt;A política RLS da tabela `licenses` permite que qualquer usuário autenticado insira registros diretamente com `payment\_status` em `Aguardando`. O frontend possui um bloco de fallback em `PlansPage.tsx:128-139` que faz `INSERT` direto com valores de `value\_paid` e `classes\_amount` controlados pelo cliente. Um atacante pode enviar requisição PostgREST direta e criar pedidos de 100 turmas com valor de R\$ 0,01.&amp;amp;lt;/p&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 546 224 558" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Por que é explorável:&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 557 673 568" data-label="Text"&amp;amp;gt;&amp;amp;lt;p&amp;amp;gt;A tabela `licenses` aceita INSERT direto por clientes em vez de exigir a execução da RPC de cálculo de preço.&amp;amp;lt;/p&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 570 215 582" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Evidência / Código:&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 581 286 600" data-label="Text"&amp;amp;gt;&amp;amp;lt;pre&amp;amp;gt;`pages/PlansPage.tsx:127-147`&amp;amp;lt;br/&amp;amp;gt;`schema.sql:151-161`&amp;amp;lt;/pre&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="99 604 612 653" data-label="Text"&amp;amp;gt;&amp;amp;lt;pre&amp;amp;gt;// schema.sql:151-161&amp;amp;lt;br/&amp;amp;gt;CREATE POLICY "Users can insert their own licenses" ON public.licenses&amp;amp;lt;br/&amp;amp;gt;FOR INSERT TO authenticated&amp;amp;lt;br/&amp;amp;gt;WITH CHECK (&amp;amp;lt;br/&amp;amp;gt; auth.uid() = user_id&amp;amp;lt;br/&amp;amp;gt;)&amp;amp;lt;br/&amp;amp;gt;ALTER POLICY "Users can insert their own licenses" ON public.licenses&amp;amp;lt;br/&amp;amp;gt;TO (payment_status IN ('Aguardando', 'under_review') OR payment_status IS NULL)&amp;amp;lt;/pre&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="99 643 157 653" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Impacto:&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 651 784 662" data-label="Text"&amp;amp;gt;&amp;amp;lt;p&amp;amp;gt;Manipulação de valores financeiros e criação de pedidos com valores incorretos ou fraudulentos na fila de análise do administrador.&amp;amp;lt;/p&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 665 236 677" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Sugestão de Correção:&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 676 616 710" data-label="List-Group"&amp;amp;gt;&amp;amp;lt;ol&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;1. Remover a política de INSERT direto para `authenticated` na tabela `licenses`.&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;2. Forçar que todos os pedidos sejam criados unicamente através da RPC `request\_license\_order`.&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;3. Remover o bloco de fallback client-side em `PlansPage.tsx`.&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;/ol&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 713 285 724" data-label="Section-Header"&amp;amp;gt;&amp;amp;lt;b&amp;amp;gt;Critérios de Aceite Verificáveis:&amp;amp;lt;/b&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="92 724 603 754" data-label="List-Group"&amp;amp;gt;&amp;amp;lt;ul&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;- [] Requisições `POST /rest/v1/licenses` diretas retornam 403 / erro de RLS&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;- [] Pedidos de licença só podem ser criados via `request\_license\_order`&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;li&amp;amp;gt;- [] Valores de licença são sempre auditados e calculados no servidor&amp;amp;lt;/li&amp;amp;gt;&amp;amp;lt;/ul&amp;amp;gt;&amp;amp;lt;/div&amp;amp;gt;&amp;amp;lt;div data-bbox="99 762 210 773" data-label="Section-Header"&amp;amp;gt;--- FIM ISSUE 4 ---&amp;amp;lt;/div&amp;amp;gt;

--- ISSUE 5 ---

Título: [Segurança] Remover e-mails de administradores hardcoded no banco e no frontend

Labels sugeridas: security, media

Descrição:

Os e-mails dos administradores mestres estão gravados de forma estática em `schema.sql:29, 36, 40` e em `components/Layout.tsx:72`. Isso expõe a identidade dos administradores no código-fonte e bundle JavaScript entregue ao usuário, além de dificultar o gerenciamento e revogação de acessos.

Por que é explorável:

Os e-mails constam como literais de string na função `is\_admin()` e no layout do frontend.

Evidência / Código:

```
`schema.sql:29` - `OR (v_user_email IN ('horium.app@gmail.com', 'prof.jackson@gmail.com'))`;`  
`components/Layout.tsx:72` - `const isMasterAdmin = ['horium.app@gmail.com',  
'prof.jackson@gmail.com'].includes(userEmailLower);`
```

```
// schema.sql:29  
OR (v_user_email IN ('horium.app@gmail.com', 'prof.jackson@gmail.com'));
```

Impacto:

Exposição pública da lista de administradores, favorecendo ataques de phishing direcionado e ataques de engenharia social.

Sugestão de Correção:

1. Remover os e-mails literais da função `is\_admin()` no PostgreSQL, consultando exclusivamente a tabela `admin\_users`.
2. Remover a verificação estática de e-mails em `Layout.tsx`, utilizando a RPC `is\_current\_user\_admin()` para determinar o papel do usuário logado.

Critérios de Aceite Verificáveis:

- [] Nenhum e-mail de administrador aparece no bundle gerado pelo Vite
- [] A função `is\_admin()` depende unicamente de registros na tabela `admin\_users`
- [] O frontend consulta o status de administrador através da RPC oficial

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

Título: [Segurança] Adicionar padrões .env e .env.* ao .gitignore para prevenir commit acidental de segredos

Labels sugeridas: security, baixa

Descrição:

O arquivo `.gitignore` atualmente inclui apenas o padrão `\*.local`. Caso desenvolvedores criem arquivos de ambiente padrão como `.env` ou `.env.production` contendo credenciais de serviços externos ou chaves de serviço do Supabase, o Git rastreará esses arquivos automaticamente.

Por que é explorável:

O padrão `.gitignore` não protege nomes de arquivo de variáveis de ambiente sem o sufixo `.local`.

Evidência / Código:

```
`gitignore:13` - `*.local`
```

```
// .gitignore:13  
*.local
```

Impacto:

Risco de versionamento acidental de chaves secretas em repositórios remotos.

Sugestão de Correção:

Adicionar as linhas `.env`, `.env.\*` e `!.env.example` ao arquivo `.gitignore`.

Critérios de Aceite Verificáveis:

- [] Arquivos `.env` e `.env.production` são ignorados pelo `git status`
- [] Apenas `.env.example` (se existente) pode ser rastreado

--- FIM ISSUE 6 ---

